

```
marp: true theme: default paginate: true style: | section { font-family: 'Segoe UI', sans-serif; background: #ffffff; color: #1a1a2e; }
section.lead { background: #1a1a2e; color: #ffffff; text-align: center; } section.lead h1 { font-size: 2.4rem; color: #e0e0ff; } section.lead
p { color: #aaaacc; font-size: 1rem; } section.modulo { background: #f0f4ff; } h1 { color: #1a1a2e; border-bottom: 3px solid #4a90d9;
padding-bottom: 0.3rem; } h2 { color: #2d5fa6; } code { background: #eef2ff; color: #1a1a2e; padding: 2px 6px; border-radius: 4px; }
table { width: 100%; border-collapse: collapse; font-size: 0.85rem; } th { background: #1a1a2e; color: #ffffff; padding: 6px 10px; } td {
border: 1px solid #dde; padding: 5px 10px; } tr:nth-child(even) td { background: #f5f7ff; } ul li { margin-bottom: 0.3rem; } .badge {
display: inline-block; background: #4a90d9; color: white; padding: 2px 8px; border-radius: 12px; font-size: 0.75rem; margin-right: 4px;
}
}
```

AtaxChile Backend

Sistema de Registro de Pacientes con Ataxia

REST API · NestJS · TypeScript · TypeORM · PostgreSQL

Objetivo del Sistema

Plataforma de gestión para la **Agrupación AtaxChile** que permite:

- Registro y seguimiento de **socios** (pacientes y representantes)
- Clasificación del **tipo de ataxia** por catálogo controlado
- Generación de **estadísticas agregadas** poblacionales en Chile
- Gestión de **usuarios administrativos** con control de acceso por roles
- Trazabilidad completa mediante **auditoría inmutable**

El sistema **no** contempla ficha médica, tratamientos ni prescripciones.

Stack Tecnológico

Capa	Tecnología
Framework	NestJS 11 + TypeScript
ORM	TypeORM 0.3
BD Producción	PostgreSQL (Supabase)
BD Desarrollo	PostgreSQL 16 vía Docker Compose (puerto 5434)
Infraestructura local	Docker Compose — PostgreSQL 16 (puerto 5434)
Autenticación	JWT (Passport) — access + refresh tokens
Seguridad HTTP	helmet (headers) + @nestjs/throttler (rate limiting)
Validación	class-validator + class-transformer

Capa	Tecnología
Email	Resend
Documentación	@nestjs/swagger + Swagger UI
Testing	Jest + Supertest

Arquitectura de Módulos

src/	
├─ auth/	Autenticación JWT, guards, decoradores
├─ users/	Usuarios administrativos (staff y directivos)
├─ geo/	Regiones y comunas de Chile (datos de referencia)
├─ miembros/	Socios de la agrupación
├─ ataxia-types/	Catálogo controlado de tipos de ataxia
├─ diagnostico-clinico/	Diagnóstico clínico por miembro (1:1, mutable)
├─ evaluacion-funcional/	Evaluaciones SARA y movilidad (append-only)
├─ stats/	Estadísticas agregadas (solo lectura)
├─ audit/	Registro de auditoría inmutable
├─ exports/	Exportación de datos (CSV/XLSX/PDF)
├─ mailer/	Envío de emails (Resend)
├─ middlewares/	Middlewares personalizados
└─ common/	TransformInterceptor, PaginatedResult, RutValidator

Separación clave: `users` (acceso al sistema) ≠ `members` (socios registrados)

Autenticación — Estrategia JWT dual

Token	Duración	Almacenamiento
<code>access_token</code>	15 minutos	Solo en cliente
<code>refresh_token</code>	7 días	Hasheado en BD

El `refresh_token` se almacena **hasheado** en `users.refreshToken`. La app falla al iniciar si faltan `JWT_SECRET` o `JWT_REFRESH_SECRET`.

Control de Acceso — Roles (RBAC)

Rol	Permisos
<code>SUPERADMIN</code>	Bypass global — gestión completa del sistema y usuarios
<code>ADMIN</code>	Miembros, catálogos, reportes y lectura completa
<code>SECRETARIO</code>	Crear, actualizar miembros, diagnósticos y evaluaciones
<code>TESORERO</code>	Lectura de miembros, cuotas y estadísticas de resumen

Rol	Permisos
USUARIO	Rol por defecto al registrarse — sin acceso administrativo

El primer usuario registrado recibe automáticamente el rol `SUPERADMIN`. `SUPERADMIN` tiene `bypass global` en `RolesGuard` — acceso implícito a todos los endpoints.

Flujos de Autenticación

Login y sesión persistente

`POST /auth/login` → `access_token` (15 min) + `refresh_token` (7 días)
`POST /auth/refresh` → nuevo `access_token` sin volver a loguearse
`POST /auth/logout` → invalida el `refresh_token` en BD

Recuperación de contraseña

`POST /auth/forgot-password` → genera `token`, envía email (respuesta genérica)
`POST /auth/reset-password` → valida `token`, hashnea nueva contraseña con `bcrypt`
`POST /auth/change-password` → cambia contraseña conociendo la actual

Registro de Cuenta — Pasos 1 a 6

Ruta: `POST /users/register` (pública)

1. Cliente envía: `nombre`, `email`, `password`
↓
2. Backend verifica que el email **no esté** registrado
→ `409` si ya existe
↓
3. Hashea `password` con `bcrypt` (10 rounds)
↓
4. Asigna `rol = 'usuario'` automáticamente
(**no se** acepta `rol` en el `body`)
↓
5. Genera **token de activación** con `randomUUID()`
↓
6. Almacena `token + expiración (24 h)` en BD
(`users.tokenActivacion`)

Registro de Cuenta — Pasos 7 a 11

Ruta de activación: `GET /users/activar/:token` (pública)

```
7. Envía email con enlace: /activar?token=<token>
  ↓
8. Responde: "Hemos enviado un correo a {email}
   para activar la cuenta"
  ↓
9. Usuario hace clic → GET /users/activar/:token
  ↓
10. Backend valida token + expiración
    → activa cuenta (cuentaActivada = true)
    ↓
11. Limpia tokenActivacion y tokenExpiracion de la BD
```

Hasta que la cuenta **no esté activada**, el login es rechazado.

Recuperación de Contraseña — Paso 1

Ruta: `POST /auth/forgot-password` (pública)

```
POST /auth/forgot-password { email }
→ Busca usuario por email
→ Genera token con crypto.randomBytes(32)
→ Almacena HASH del token + expiración (1 hora) en BD
→ Envía email con enlace al frontend:
  /reset-password?token=<token-en-claro>
→ Responde SIEMPRE con mensaje genérico
  (no confirma si el email existe →
   previene enumeración de usuarios)
```

Recuperación de Contraseña — Paso 2

Ruta: `POST /auth/reset-password` (pública)

```
POST /auth/reset-password { token, nuevaPassword }
→ Busca usuario por HASH del token recibido
→ Verifica que el token no haya expirado
  (400 si expiró)
→ Hashea la nueva contraseña con bcrypt (10 rounds)
→ Guarda nueva contraseña en BD
→ Limpia resetPasswordToken y resetPasswordExpires
→ Registra evento en auditoría
```

Módulo Geo

Regiones y Comunas de Chile

- Datos de referencia **estáticos** — 16 regiones + 346 comunas
- Poblados automáticamente con un **seeder** al iniciar la app (solo si las tablas están vacías)

- Al crear un miembro se envía `comunaId` ; la **región se obtiene** vía `comuna → region`

No se implementa `DELETE` — la eliminación rompería FKs de miembros existentes.

Módulo Geo — Endpoints

Método	Ruta	Acceso
GET	<code>/geo/regiones</code>	Público
GET	<code>/geo/regiones/:id/comunas</code>	Público
POST	<code>/geo/regiones</code>	superadmin, admin
PATCH	<code>/geo/regiones/:id</code>	superadmin, admin
POST / PATCH	<code>/geo/comunas</code>	superadmin, admin

Módulo Tipos de Ataxia

Catálogo Controlado

- 59 tipos** clasificados en 4 grupos
- Migrado desde el sistema legado SQL (`tipo_ataxia`) y enriquecido con terminología clínica actual

Grupo	Descripción	Ejemplos
<code>hereditaria</code>	Origen genético	Friedreich, SCA1–SCA30, DRPLA, FXTAS
<code>adquirida</code>	Causas externas	MSA-C, alcohólica, paraneoplásica, CANVAS
<code>idiopatica</code>	Sin causa identificada	SAOA, esporádica no clasificada
<code>otra</code>	En investigación	Origen combinado, sin diagnóstico

La confirmación diagnóstica (`genetico` / `clinico` / `probable`) pertenece a `DiagnosticoClinico` , no al catálogo.

Seeder de Tipos de Ataxia

Migración desde tabla SQL legacy (`tipo_ataxia`) al nuevo seeder TypeScript: **32 registros** previos → **59 registros** en el seeder actualizado

Origen	Destino	Acción
<code>SCA1 – SCA30</code> (con gaps)	<code>HEREDITARIA</code>	Incorporados con nombre clínico
<code>EA1 – EA7</code>	<code>HEREDITARIA</code>	Completados (SQL solo tenía 7)

Origen	Destino	Acción
DRPLA , FXTAS , MIRAS , MSS	HEREDITARIA	Incorporados con descripción
ATAXIA CEREBELOSА ADQUIRIDA	ADQUIRIDA	Incorporado
DESCONOCIDA	OTRA	Renombrado con descripción
MSA-C, CANVAS, AVED, etc.	Nuevo sistema	Agregados (no estaban en SQL)

Módulo Miembros

Perfiles diferenciados

Campo	Paciente	Representante
tipoAtaxiaId	Referencia al catálogo	null
diagnosticoClinico	Relación 1:1	null
tipoRepresentacion	null	Obligatorio
representadoId / representadoNombre	null	Al menos uno

El diagnóstico completo vive en `DiagnosticoClinico` , separado del miembro.

Módulo Miembros — Campos Clave (1/2)

Campo	Tipo	Descripción
rut	string único	RUT chileno validado
nombre	string	Nombre completo
celular	string nullable	Teléfono móvil
profesion	string nullable	Profesión u oficio
estadoCivil	enum	soltero / casado / viudo / divorciado

Módulo Miembros — Campos Clave (2/2)

Campo	Tipo	Descripción
estado	enum	activo / renunciado / suspendido / fallecido
fechaCambioEstado	Date nullable	Se actualiza automáticamente

Campo	Tipo	Descripción
<code>representadoNombre</code>	<code>string nullable</code>	Nombre del representado (si no está registrado)
<code>representadoRut</code>	<code>string nullable</code>	RUT del representado (si no está registrado)

`anio_inscripcion` y `anio_cambio_estado` no se almacenan — se derivan con `.getFullYear()`.

Módulo Diagnóstico Clínico

Entidad `DiagnosticoClinico` — relación 1:1 con `Miembro`

Campo	Tipo	Descripción
<code>tipoAtaxiaId</code>	FK → <code>AtaxiaType</code>	Tipo de ataxia del catálogo
<code>subtipo</code>	<code>string nullable</code>	Texto libre (ej: SCA2, Friedreich)
<code>confirmacion</code>	<code>ConfirmacionDiagnostico</code>	<code>genetico</code> / <code>clinico</code> / <code>probable</code>
<code>fechaDiagnostico</code>	<code>date nullable</code>	Fecha del diagnóstico
<code>institucion</code>	<code>string nullable</code>	Centro donde se diagnosticó
<code>medico</code>	<code>string nullable</code>	Médico tratante
<code>observaciones</code>	<code>text nullable</code>	Notas adicionales

Solo los pacientes (`esRepresentante: false`) tienen diagnóstico clínico asociado.

Módulo Evaluación Funcional

Entidad `EvaluacionFuncional` — append-only

Campo	Tipo	Descripción
<code>fecha</code>	<code>date</code>	Fecha de la evaluación
<code>nivelMovilidad</code>	<code>NivelMovilidad</code>	ambulatorio / silla parcial / silla total / postrado
<code>puntuacionSara</code>	<code>int nullable</code>	Escala SARA 0–40 (estándar internacional)
<code>disartria</code>	<code>boolean</code>	Dificultad para hablar
<code>disfagia</code>	<code>boolean</code>	Dificultad para tragar
<code>nistagmo</code>	<code>boolean</code>	Movimiento involuntario de ojos
<code>tieneCuidador</code>	<code>boolean</code>	Tiene cuidador asignado
<code>registradoPorId</code>	FK → <code>User</code>	Usuario que registró la evaluación

Append-only: sin UPDATE ni DELETE. Cada evaluación es un hecho histórico inmutable.

Representación de Socios

Un **representante** es un familiar, tutor o cuidador de una persona con ataxia.

```
export enum TipoRepresentacion {  
  PADRE_MADRE = 'padre_madre',  
  CONYUGE = 'conyuge',  
  HIJO_HIJA = 'hijo_hija',  
  TUTOR_LEGAL = 'tutor_legal',  
  CUIDADOR = 'cuidador',  
  OTRO = 'otro',  
}
```

- Si la persona representada **está registrada** → `representadoId` (UUID, relación auto-referencial)
- Si **no está registrada** → `representadoNombre` + `representadoRut` (campos de texto)
- Los representantes se **excluyen** de las estadísticas de diagnóstico de ataxia

API REST — Endpoints por Módulo (1/2)

Módulo	Base	Operaciones
Auth	<code>/auth</code>	login, refresh, logout, forgot/reset/change-password
Usuarios	<code>/users</code>	CRUD + activar cuenta + cambio de rol/status
Geo	<code>/geo</code>	Regiones y comunas (lectura pública, escritura admin)
Tipos de Ataxia	<code>/ataxia-types</code>	Catálogo con filtro <code>?grupo=</code> + soft delete
Miembros	<code>/miembros</code>	<code>POST</code> , <code>GET</code> , <code>GET /:id</code> , <code>PATCH /:id</code> , <code>PATCH /:id/estado</code> , <code>PATCH /:id/vincular-usuario</code>
Diagnóstico Clínico	<code>/miembros/:id/diagnostico</code>	<code>POST</code> , <code>GET</code> , <code>PATCH</code>
Eval. Funcional	<code>/miembros/:id/evaluaciones</code>	<code>POST</code> (append-only), <code>GET</code> , <code>GET /ultima</code>

Base URL: `/api/v1`

API REST — Endpoints por Módulo (2/2)

Módulo	Base	Operaciones
Estadísticas	<code>/stats</code>	resumen · miembros · diagnósticos · funcional · geográfico · cuotas*
Auditoría	<code>/audit-logs</code>	Solo lectura, acceso SUPERADMIN

Módulo	Base	Operaciones
Exportaciones	/exports	CSV / XLSX — solo roles autorizados

* pendiente módulo cuotas

Documentación Swagger

Configuración aplicada

```
// main.ts
const config = new DocumentBuilder()
  .setTitle('AtaxChile API')
  .setDescription('Sistema de registro de miembros de la Asociación AtaxChile')
  .setVersion('1.0')
  .addBearerAuth()
  .build();
SwaggerModule.setup('api/docs', app, document);
```

UI disponible en: `http://localhost:5000/api/docs`

Cobertura

- ✓ `@ApiProperty` en todos los DTOs con ejemplos reales
- ✓ `@ApiTags` agrupa endpoints por módulo
- ✓ `@ApiBearerAuth` en rutas protegidas
- ✓ `@ApiOperation` + `@ApiResponse` en cada endpoint
- ✓ `PartialType` migrado de `@nestjs/mapped-types` → `@nestjs/swagger`

Módulo Stats

Estadísticas agregadas — solo lectura

Endpoint	Roles	Descripción
GET /stats/resumen	ADMIN, SEC, TES	Totales y variaciones recientes
GET /stats/miembros	ADMIN, SECRETARIO	Distribución por estado
GET /stats/diagnosticos	ADMIN, SECRETARIO	Por tipo de ataxia y confirmación
GET /stats/funcional	ADMIN, SECRETARIO	Última evaluación por miembro
GET /stats/geografico	ADMIN, SECRETARIO	Distribución por región

- Sin entidad propia — agrega datos con `QueryBuilder` + `GROUP BY`
- Representantes excluidos de diagnósticos, funcional y geográfico

Módulo Audit

Registro inmutable de eventos del sistema

Campo	Tipo	Descripción
accion	enum	LOGIN · CREAR_MIEMBRO · MODIFICAR_MIEMBRO · ...
entidad	varchar	Tabla afectada (miembros , users , etc.)
entidadId	varchar null	ID del registro afectado
detalle	json null	Campos cambiados u otros datos del evento
usuarioId	FK null	Quién ejecutó la acción
ip	varchar null	IP del cliente

- **Append-only** — sin UPDATE ni DELETE
- `AuditService` es interno — los servicios lo inyectan; no hay endpoint de creación
- Fallo en auditoría no interrumpe la operación principal (fire-and-forget)
- Lectura restringida exclusivamente a SUPERADMIN

Módulo Audit — Endpoints

Método	Ruta	Acceso	Descripción
GET	/audit-logs	SUPERADMIN	Lista paginada con filtros
GET	/audit-logs/:id	SUPERADMIN	Detalle de un evento específico

Filtros disponibles en `GET /audit-logs` :

- `?page=1&limit=20` — paginación
- `?accion=LOGIN` — por tipo de acción
- `?entidad=miembro` — por entidad afectada
- `?usuarioId=5` — por usuario
- `?desde=2024-01-01&hasta=2024-12-31` — rango de fechas

Módulo Exports

Exportación de datos — CSV / XLSX / PDF

Campo	Tipo	Descripción
formato	enum	csv · xlsx · pdf

Campo	Tipo	Descripción
estado	string	Filtrar por estado del socio
regionId	number	Filtrar por región
tipoAtaxiaId	number	Filtrar por tipo de ataxia
fechaDesde	ISO date	Filtrar por fecha de ingreso
fechaHasta	ISO date	Filtrar por fecha de ingreso

Módulo Exports — Endpoints

Método	Ruta	Acceso	Descripción
GET	/exports/miembros	ADMIN, SECRETARIO, TESORERO	Exportar lista de miembros
GET	/exports/miembros/:id/evaluaciones	ADMIN, SECRETARIO	Ficha individual con historial (PDF)

Formatos disponibles: [?formato=csv](#) · [?formato=xlsx](#) · [?formato=pdf](#)

Columnas por rol:

- ADMIN / SECRETARIO: todos los campos incluyendo diagnóstico y evaluaciones
- TESORERO: columnas reducidas (sin datos clínicos sensibles)

Estado Actual del Proyecto

Módulo	Estado
auth	✓ Implementado y testeado
users	✓ Implementado y testeado
geo	✓ Implementado y testeado
ataxia-types	✓ Implementado y testeado
miembros	✓ Implementado y testeado
diagnostico-clinico	✓ Implementado y testeado
evaluacion-funcional	✓ Implementado (append-only) y testeado
stats	✓ Implementado (solo lectura, sin entidad propia) y testeado
audit	✓ Implementado (append-only) y testeado
exports	✓ Implementado (CSV/XLSX/PDF) y testeado

Módulo	Estado
Swagger	✅ Configurado en todos los módulos implementados
Docker Compose	✅ PostgreSQL 16 local (puerto 5434)
Seguridad	✅ helmet + throttler configurados

Próximos Pasos

- 1. Implementar módulo `cuotas` (TarifaAnual + Cuota, diseño documentado) y además todos los módulos que sean parte de finanzas queda para otras etapas de este proyecto por decisión de la directiva de la asociación.
- 2. Migraciones TypeORM para paso a producción
- 3. Tests e2e para módulos de diagnóstico, evaluación y stats

AtaxChile Backend

Repositorio: `backend/` Documentación técnica: `docs/api-design.md` Guía para agentes IA: `AGENTS.md` Swagger UI: `http://localhost:5000/api/docs`

Agrupación de Pacientes con Ataxia – Chile